

Relatório do Projeto - Farmpedia

Disciplina: CIN0135 — Estruturas de Dados Orientadas a Objetos (CIn/UFPE, 2026.2)

Equipe: Gabriela Leite de Andrade Lima | Kamilla Regina Menezes de Souza | Sofia Sobreira de Mendonça

1. Visão Geral do Projeto

O **Farmpedia** é um sistema de gerenciamento de fazenda e databank interativo desenvolvido em C++17, inspirado no universo do jogo *Stardew Valley*. O objetivo principal do projeto é prover uma ferramenta para auxiliar os jogadores a navegarem pelas diversas mecânicas do jogo, ao mesmo tempo em que permite registrar, consultar e atualizar o progresso individual de sua própria fazenda em um banco de dados SQLite3.

Stardew Valley abrange uma vasta quantidade de conteúdo: 34 NPCs interativos com preferências únicas de presentes e rotinas, 54 tipos de hortaliças com especificidades de época de cultivo e preços de mercado, dezenas de animais de fazenda, construções e missões secundárias. Diante disso, a experiência de jogo pode se tornar desafiadora e exaustiva para o gerenciamento de informações. O Farmpedia resolve esse problema atuando como uma enciclopédia interativa organizada em cinco "livros" de consulta e um gerenciador ativo da fazenda do jogador.

2. Arquitetura do Sistema

O Farmpedia foi organizado de forma modular, separando a interface do usuário, a lógica do sistema e o acesso aos dados. Essa organização facilita a manutenção e permite que diferentes partes do sistema sejam desenvolvidas de forma independente.

2.1 Componentes Principais

- **Interface de Usuário (CLI / Protótipo Web):** O sistema possui uma interface de linha de comando (CLI) interativa via terminal para navegação entre menus, filtros e cadastros, além de um protótipo de interface gráfica Web localizado na pasta gui/.
- **Núcleo do Sistema em C++ (Lógica de Negócios):** Implementa os conceitos de Programação Orientada a Objetos para modelar as entidades de referência do jogo (Crops, Villagers, Animals e Buildings) e o domínio da fazenda do jogador (MyFarm, MyAnimal, MyBuilding e MyRelationship). Também contém funcionalidades de busca e filtragem, como busca por nome e filtro de crops por estação.
- **Acesso aos dados (SQLite3 Embutido):** Utiliza a biblioteca SQLite3 em linguagem C (external/sqlite3.c) compilada diretamente junto com o projeto, eliminando a necessidade

de dependências de ambiente ou instalações externas adicionais.

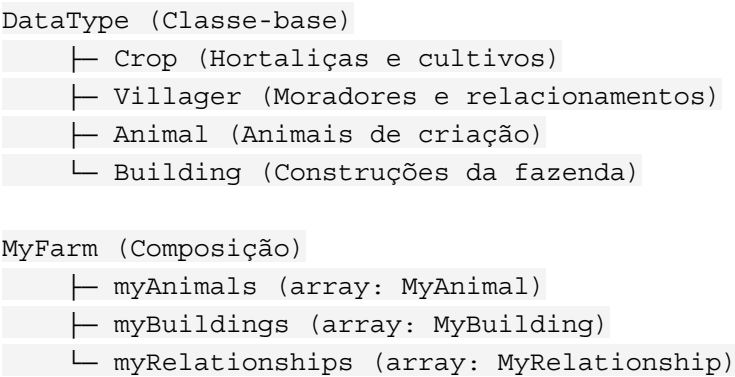
2.2 Fluxo de Dados

O fluxo de dados no Farmpedia envolve a entrada do usuário pela CLI/GUI, o processamento pelos módulos de menus e filtros, a manipulação dos objetos em C++ e o acesso aos bancos SQLite3. As consultas do catálogo utilizam os dados de referência, enquanto as operações relacionadas à “My Farm” também permitem armazenar e atualizar informações da fazenda do jogador.

3. Implementação POO em C++

A implementação do backend em C++17 fundamenta-se nos quatro pilares essenciais da Programação Orientada a Objetos. A classe-base `DataType` unifica as entidades de consulta, enquanto a classe `MyFarm` gerencia a propriedade do jogador por composição.

3.1 Hierarquia de Classes e Estrutura



3.2 Conceitos de POO Aplicados

A tabela abaixo resume onde e como cada conceito de POO foi aplicado na arquitetura do projeto:

Conceito POO	Onde foi usado	Descrição e Aplicação Prática
Classes e Objetos	Crop, Villager, Animal, Building, MyFarm, MyAnimal, MyBuilding, MyRelationship	Cada entidade possui classe dedicada; registros do banco de dados são instanciados como objetos em memória para manipulação.

Conceito POO	Onde foi usado	Descrição e Aplicação Prática
Herança	DataType → Crop, Villager, Animal, Building	DataType é a classe-base das entidades de referência do jogo. A partir dela são derivadas as classes Crop, Villager, Animal e Building, compartilhando atributos básicos como nome e tipo.
Composição	MyFarm	a classe MyFarm representa a fazenda do jogador e mantém coleções próprias de MyAnimal, MyBuilding e MyRelationship, em vez de herdar de DataType
Polimorfismo	virtual ~DataType(), métodos virtuais nas classes do catálogo	O destrutor virtual permite que objetos derivados de DataType sejam destruídos corretamente quando manipulados por ponteiros para a classe-base.
Modificadores de Acesso	private e public	Os atributos das classes são mantidos privados para favorecer o encapsulamento, enquanto métodos públicos, como getters e setters, permitem o acesso controlado aos dados.
Ponteiros e Referências	sqlite3* db, const std::string&	O projeto utiliza ponteiros, como sqlite3* para a conexão com o banco e DataType* na busca geral, além de referências constantes (const std::string&) para evitar cópias desnecessárias de strings.

4. Banco de Dados e Persistência

O projeto utiliza dois bancos SQLite com funções distintas. O **gameData.db** concentra os dados de referência do jogo, utilizados para consulta no catálogo, enquanto o **myFarmData.db** armazena os dados personalizados da fazenda do jogador e é utilizado para operações de leitura e escrita.

Essa separação permite distinguir os dados estáticos do jogo dos dados que representam o progresso e as informações específicas da fazenda.

5. Estrutura dos 5 Livros do Farmpedia

O sistema é estruturado funcionalmente em cinco livros interativos:

1. **Livro 1 — Crops:** Exibe a estação recomendada, dias até a colheita, rebrota, preços de venda por qualidade, preços de sementes e produtos artesanais derivados.
2. **Livro 2 — Villagers:** Exibe catálogo de moradores, presentes (amados, gostados, neutros e odiados), rotina diária e histórico de corações.
3. **Livro 3 — Animals:** Mapeia espécies de animais, produtos gerados, dias até atingir a fase adulta, preço de compra/venda e itens artesanais derivados.
4. **Livro 4 — Buildings:** Armazena especificações das construções, lista de materiais exigidos, custo em ouro, dimensões do grid e capacidade de abrigar animais.
5. **Livro 5 — My Farm (Gerenciador do Jogador):** Módulo interativo que suporta as operações CRUD completas:
 - **Create:** Registra novos animais, construções e relacionamentos na fazenda.
 - **Read:** Apresenta um relatório/resumo estruturado do estado atual da propriedade.
 - **Update:** Atualiza dados dinâmicos, tais como nível de afeição e corações com NPCs.
 - **Delete:** Remove registros (ex: edifícios demolidos ou animais vendidos).

6. Estrutura de Arquivos e Compilação

A organização dos diretórios do projeto reflete a separação de código, dados e dependências:

```
src/
├─ main.cpp           → Ponto de entrada do programa
├─ database.cpp       → Camada de manipulação SQL e conexão SQLite3
├─ filters.cpp        → Módulo de ordenação e filtros de busca
├─ menu.cpp           → Menus interativos no terminal
├─ classes/           → Declaração das classes POO (DataType, Crop, MyFarm,
etc.)
```

- |— definitions.cpp → Implementação dos métodos das classes
- |— menus/ → Headers dos filtros e menus de navegação
- data/ → Arquivos de banco de dados (gameData.db, myFarmData.db)
- external/ → Dependência SQLite3 em C (sqlite3.c, sqlite3.h)
- gui/ → Protótipo da interface gráfica Web

Comandos para Compilação e Execução:

```
# Linux / macOS
gcc -c external/sqlite3.c -o sqlite3.o
g++ -std=c++17 src/main.cpp src/database.cpp src/filters.cpp src/menu.cpp
src/definitions.cpp sqlite3.o -o farm_manager
./farm_manager

# Windows (PowerShell)
gcc -c external/sqlite3.c -o sqlite3.o
g++ -std=c++17 src/main.cpp src/database.cpp src/filters.cpp src/menu.cpp
src/definitions.cpp sqlite3.o -o farm_manager.exe
.\farm_manager.exe
```

7. Autores e Contribuições

O projeto Farmpedia foi desenvolvido em grupo para a disciplina CIN0135 (UFPE), com a seguinte divisão de responsabilidades:

Integrante	Atribuições e Contribuições no Projeto
Gabriela Leite de Andrade Lima	Estruturação do diagrama e esquema do banco de dados, implementação da classe DataType, conexão e acesso ao banco, implementação de MyFarm e MyAnimal, criação do banco da fazenda, CRUD conectado ao banco e seus testes, menu principal, telas de consulta da CLI, testes de integração e configuração/implementação das telas em Dear ImGui.
Kamilla Regina Menezes de Souza	Implementação das classes Animal e Building e da classe MyRelationship, além da participação na implementação e nos testes dos sistemas de leitura e filtragem.

Integrante	Atribuições e Contribuições no Projeto
Sofia Sobreira de Mendonça	Implementação das classes Crop, Villager e MyBuilding, participação na implementação e nos testes dos sistemas de leitura e filtragem, além da implementação do protótipo da GUI Web.